Deliverable Sprint 1 — Event Registration System

Course: Software Engineering / Agile Architecture & Design (A4D)
Date: November 8, 2026
Repository: https://github.com/ImanolLabarrere/A4D-Event-Registration
Release Tag: `v0.1.0-sprint1` (commit `ab9936e`)
Team Members:

- **Travailleur A:** gobert (`hoshiiiix`)
- **Travailleur B:** Imanol Labarrere (`ImanolLabarrere`)

Table of Contents

1. Part A: Context, Actors, Scope & Assumptions
2. Part B: Software Requirements Specification (SRS)
3. Part C: Software Design Specification (SDS)
4. Part D: Sprint 1 Plan, Contracts & Definition of Done
5. Part E: Traceability Matrix
6. Part F: Verification, Testing & Evidence

Part A: Context, Actors, Scope & Assumptions

1. Context and Problem

A university department organizes workshops, technical seminars, and conferences with strictly limited seating capacities. Historically, registrations were collected through informal channels (emails, chat messages, manual spreadsheets), leading to frequent overbooking, duplicate registrations, concurrency conflicts, and lack of visibility for organizers.

The project objective is to deliver a robust, concurrency-safe, role-governed web application ensuring that capacity limits are strictly enforced under concurrent traffic, data survives container restarts, and user privacy is preserved.

2. Actors and Roles

- **Organizer:** Creates events as drafts, publishes them once finalized (with a future date and positive capacity), and inspects the list of confirmed attendees for their own events.

- **Participant:** Browses upcoming published events, checks remaining available seats, registers for events within capacity limits, and manages their own registrations.
- **Anonymous Visitor:** Cannot access protected resources; receives standard `401 UNAUTHENTICATED` responses without leakage of internal data.

3. Scope of Sprint 1

- **F-01 Authentication:** Session-based authentication using Argon2id password hashing, HTTP-only cookie session management, and server-side role verification.
- **F-02 Create and Publish Events:** Draft event creation and publication rules (future date required, positive capacity).
- **F-03 Browse Published Events:** Public listing of upcoming published events with accurate `confirmedCount` and `remainingPlaces`, and organizer-isolated views.
- **F-04 Register for an Event:** Concurrency-safe registration enforcing capacity limits, deduplication, and past-event rejection via database row-level locking (`SELECT ... FOR UPDATE`).
- **F-05 My Registrations:** Participant-isolated registration consultation (`GET /api/registrations`, `GET /api/registrations/:id`).
- **F-06 Organizer View of Registrations:** Organizer-restricted attendee list for owned events (`GET /api/events/:id/registrations`).

Excluded from Sprint 1 (deferred to Sprints 2 & 3): Cancellations (F-07), Waiting list (F-08), Event editing (F-09), Registration deadlines (F-10), Check-in (F-11), Attendance reports (F-12).

4. Assumptions

- **A-01:** Server time is synchronized (UTC).
- **A-02:** All sensitive credentials (`DATABASE_URL`, `SEED_PASSWORD`) are injected via environment variables; none are committed to version control.
- **A-03:** Accounts are seeded for demo/testing purposes (no self-registration in Sprint 1).
- **A-04:** PostgreSQL is used with full ACID transactions and row-level locking capabilities.

Part B: Software Requirements Specification (SRS)

1. Functional Requirements (Sprint 1)

ID	Requirement	Feature
FR-01	User logs in with email/password, logs out, and fetches current identity	F-01
FR-02	Organizer creates an event (title, date, capacity) in DRAFT status	F-02
FR-03	Organizer publishes an event with a date strictly in the future	F-02

ID	Requirement	Feature
FR-04	Participant lists upcoming published events with remaining seats	F-03
FR-05	Participant registers for an open published event	F-04
FR-06	Participant views own registrations; Organizer views attendees of own event	F-05, F-06

2. Business Rules (Sprint 1)

ID	Rule Description	Enforcement
BR-01	Event capacity must be a strictly positive integer (`capacity > 0`).	DB `CHECK` constraint & Zod validation
BR-02	An event can only be published if its start date is strictly in the future.	Service rule `canPublish` & endpoint guard
BR-03	Total confirmed registrations cannot exceed event capacity.	DB transaction with row lock
BR-04	A participant cannot register more than once for the same event.	Unique constraint / existence check
BR-05	Registration is rejected if the event has already started.	Service rule `decideRegistration`
BR-06	Priority when evaluating registration: `PAST` > `DUPLICATE` > `FULL` .	`decideRegistration` order
BR-07	An organizer can only publish or view registrations for events they own.	Query filter `organizerId: req.user.id`
BR-09	A participant can only access their own registrations.	Query filter `userId: req.user.id`

3. Non-Functional Requirements (NFR)

- **NFR-01 (Authorization):** Server-side authentication and authorization enforced on all protected endpoints (`requireAuth` , `requireRole`).
- **NFR-02 (Password Storage):** Passwords hashed using Argon2id (timing-safe verification, dummy hash on invalid accounts).
- **NFR-03 (Concurrency):** Concurrent registrations for the last seat serialized via database row lock (`SELECT ... FOR UPDATE`); exactly 1 success under race conditions.
- **NFR-04 (Persistence):** Data survives API and database container restarts without loss (verified via Docker volume).
- **NFR-05 (Secrets Isolation):** No `.env` file or credentials tracked in Git repository (verified via `git ls-files`).
- **NFR-06 (Session TTL):** Sessions expire after 8 hours; expired sessions are rejected.

- **NFR-07 (Privacy):** No sensitive data (password hashes, emails of other users, internal IDs) leaked in public API responses; uniform error structure (`D-08`).

Part C: Software Design Specification (SDS)

1. Architecture Overview

The system follows a modular 3-tier architecture:

- **Presentation / Client:** End-to-end user journeys validated through Playwright (E2E suite).
- **Application Server:** Node.js 22 LTS, Express, TypeScript, Zod schema validation, Argon2, Supertest.
- **Persistence Layer:** PostgreSQL 16 container, Prisma ORM, automated migrations, named Docker volume `pgdata` .

2. Data Model (Prisma Schema)

```
model User {
  id            String         @id @default(uuid())
  email         String         @unique
  passwordHash  String
  role          Role           // ORGANIZER, PARTICIPANT
  createdAt     DateTime       @default(now())
  events        Event[]        @relation("OrganizerEvents")
  registrations Registration[]
  sessions      Session[]
}

model Session {
  id        String   @id @default(uuid())
  token     String   @unique
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  expiresAt DateTime
  createdAt DateTime @default(now())
}

model Event {
  id            String         @id @default(uuid())
  title         String
  startsAt      DateTime
  capacity      Int
  status        EventStatus    @default(DRAFT) // DRAFT, PUBLISHED
  organizerId   String
  organizer     User           @relation("OrganizerEvents", fields: [organizerId], references: [
  registrations Registration[]
  createdAt     DateTime       @default(now())
}
```

```
model Registration {
  id        String             @id @default(uuid())
  eventId   String
  userId    String
  status    RegistrationStatus @default(CONFIRMED) // CONFIRMED, CANCELLED
  createdAt DateTime           @default(now())
  event     Event              @relation(fields: [eventId], references: [id], onDelete: Cascade)
  user      User               @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([eventId, userId])
}
```

3. REST API Endpoint Catalog

Method	Endpoint	Role Guard	Description
`POST`	`/api/auth/login`	Anonymous	Authenticates credentials, returns user profile, issues `sid` cookie
`POST`	`/api/auth/logout`	Authenticated	Revokes session, clears `sid` cookie
`GET`	`/api/auth/me`	Authenticated	Returns current authenticated user
`POST`	`/api/events`	`ORGANIZER`	Creates a new draft event
`POST`	`/api/events/:id/publish`	`ORGANIZER`	Publishes an event owned by the caller
`GET`	`/api/events`	Authenticated	Lists upcoming published events with remaining seats
`GET`	`/api/events/mine`	`ORGANIZER`	Lists all events owned by current organizer
`GET`	`/api/events/:id`	Authenticated	Retrieves event detail (or owned draft)
`POST`	`/api/events/:id/registrations`	`PARTICIPANT`	Registers caller for event (row-locked transaction)
`GET`	`/api/registrations`	`PARTICIPANT`	Lists caller's own registrations
`GET`	`/api/registrations/:id`	`PARTICIPANT`	Retrieves caller's single registration
`GET`	`/api/events/:id/registrations`	`ORGANIZER`	Lists attendees of owned event

Part D: Sprint 1 Plan, Contracts & Definition of Done

1. Division of Responsibilities

To ensure strict separation of concerns, peer review, and adherence to agile methodology:

- **Travailleur A (`hoshiiiix`):**

    - Contracts: `F-02-events.md`, `F-04-register.md`, `F-05-my-registrations.md`, `dod.md`.
    - Implementations: F-02 (Create & Publish), F-04 (Register with concurrency lock).
    - Documentation: Local setup & README (`PR #18`).
- **Travailleur B (`ImanolLabarrere`):**
    - Contracts: `F-01-auth.md`, `F-03-browse.md`, `F-06-organizer-view.md`.
    - Implementations: Initial skeleton & DB schema (`PR #10`), F-01 (Authentication), F-03 (Browse events), F-06 (Organizer view), E2E test suite Playwright (`PR #22`), NFR-04 & NFR-05 verifications.

2. Definition of Done (DoD v1.0) Compliance

All 9 DoD criteria have been strictly fulfilled:

1. ✅ **Contract Behavior:** Fully implemented across all 6 features.
2. ✅ **Test Coverage:** All acceptance criteria mapped to passing automated tests.
3. ✅ **TDD Discipline:** Red test commit preceding green implementation for every feature.
4. ✅ **Backend Authorization:** Enforced via `requireAuth` and `requireRole`; no client-supplied ID is trusted.
5. ✅ **D-08 Error Structure:** Uniform `{ error: { code, message } }`; zero leaks of stack traces or SQL.
6. ✅ **Peer Review:** Feature branches reviewed by the other team member; zero self-approved PRs.
7. ✅ **Release PR & Tag:** PR #23 & #24 merged into `main`; tagged `v0.1.0-sprint1`.
8. ✅ **Hygiene:** No secrets committed; `.env.example` provided.
9. ✅ **Traceability & Evidence:** Complete test runs, commit hashes, and outputs recorded.

Part E: Traceability Matrix

1. Functional Requirements Traceability

Requirement	Feature	AC IDs	Automated Tests (Vitest & Playwright)
FR-01 (Auth)	F-01	AC-101..114	`TEST-101`..`TEST-114` (`auth.test.ts`, `auth-unit.test.ts`, `journey.spec.ts`)
FR-02 (Create event)	F-02	AC-201..205	`TEST-201`..`TEST-205` (`events-manage.test.ts`, `event-rules.test.ts`)
FR-03 (Publish event)	F-02	AC-206..213	`TEST-206`..`TEST-213` (`events-manage.test.ts`, `event-rules.test.ts`, `journey.spec.ts`)
FR-04 (Browse events)	F-03	AC-301..313	`TEST-301`..`TEST-314` (`browse.test.ts`, `browse-unit.test.ts`)

Requirement	Feature	AC IDs	Automated Tests (Vitest & Playwright)
FR-05 (Register)	F-04	AC-401..412	`TEST-401` .. `TEST-412` (`register.test.ts` , `registration-rules.test.ts` , `journey.spec.ts`)
FR-06 (My registrations)	F-05	AC-501..508	`TEST-501` .. `TEST-508` (`my-registrations.test.ts`)
FR-06 (Organizer view)	F-06	AC-601..609	`TEST-601` .. `TEST-610` (`organizer-view.test.ts` , `journey.spec.ts`)

2. Business Rules Traceability

Rule	Feature	Verification Test
BR-01 (Capacity > 0)	F-02, R-02	`TEST-201` , `TEST-214` (DB check constraint)
BR-02 (Future date required)	F-02	`TEST-202` , `TEST-206` , `TEST-209`
BR-03 (Capacity limit)	F-04	`TEST-401` , `TEST-404` , `TEST-411`
BR-04 (No duplicate)	F-04	`TEST-403` , `TEST-406`
BR-05 (Past event registration rejected)	F-04	`TEST-405`
BR-06 (Priority: PAST > DUPLICATE > FULL)	F-04	`TEST-407` , `TEST-408`
BR-07 (Organizer ownership)	F-02, F-06	`TEST-208` , `TEST-213` , `TEST-603`
BR-09 (Participant ownership)	F-05	`TEST-501` , `TEST-504`

3. Non-Functional Requirements Traceability

NFR	Scope	Verification Evidence
NFR-01 (Authorization)	Global	401/403 tests on all routes (`TEST-102` , `TEST-207` , `TEST-307` , `TEST-410` , `TEST-506` , `TEST-604` , `E2E-03`)
NFR-02 (Argon2id)	F-01	`TEST-112` (verifies `$argon2id$` prefix and salt/params)
NFR-03 (Concurrency)	F-04	`TEST-411` (20 simultaneous requests on real DB for last seat $\rightarrow$ exactly 1 success)
NFR-04 (Persistence)	Database	Container restart procedure documented with row counts before/after
NFR-05 (No secrets in Git)	Repository	`git ls-files | grep '\.env'` returns only `.env.example`
NFR-06 (Session TTL)	F-01	`TEST-103` , `TEST-111` (8-hour TTL expiration check)
NFR-07 (Privacy / Safe errors)	Global	`TEST-313` , `TEST-508` , `TEST-608` (no password hash, email or internal IDs leaked)

Part F: Verification, Testing & Evidence

1. Test Suite Execution Summary

- **Unit & Integration Suite (Vitest):**
  - Test files: **11 passed / 11**
  - Tests: **101 passed / 101** (0 failed, 0 skipped)
  - Execution time: ~23s
- **End-to-End Suite (Playwright):**
  - Scenarios: **3 passed / 3** (0 failed)
  - `TEST-114` / `E2E-01`: Authentication journey (login, session validation, logout, rejection)
  - `E2E-02`: Full lifecycle (Organizer creates & publishes event, Participant registers and inspects registration)
  - `E2E-03`: Security & role boundary enforcement

2. TDD Cycle Artifacts

Every feature followed a strict Red $\rightarrow$ Green test-driven progression:

- **F-01 Authentication:** Red (`docs/evidence/F-01-red.txt`) $\rightarrow$ Green (`docs/evidence/F-01-green.txt`)
- **F-02 Create & Publish:** Red (`docs/evidence/F-02-red.txt`, `F-02-http-red.txt`) $\rightarrow$ Green (`docs/evidence/F-02-green.txt`, `F-02-http-green.txt`)
- **F-03 Browse Events:** Red (`docs/evidence/F-03-red.txt`) $\rightarrow$ Green (`docs/evidence/F-03-green.txt`)
- **F-04 Register (Concurrency):** Red (`docs/evidence/F-04-unit-red.txt`, `F-04-http-red.txt`) $\rightarrow$ Green (`docs/evidence/F-04-unit-green.txt`, `F-04-http-green.txt`)
- **F-05 My Registrations:** Red (`docs/evidence/F-05-red.txt`) $\rightarrow$ Green (`docs/evidence/F-05-green.txt`)
- **F-06 Organizer View:** Red (`docs/evidence/F-06-red.txt`) $\rightarrow$ Green (`docs/evidence/F-06-green.txt`)

3. NFR-03 Concurrency Proof (TEST-411)

To demonstrate compliance with NFR-03, a stress test was executed against a real PostgreSQL instance:

- Event capacity: `1`.
- Concurrent requests: `20` simultaneous participant registration attempts using `Promise.all`.
- Implementation: `SELECT "id" FROM "Event" WHERE "id" = $1 FOR UPDATE` inside a Prisma transaction with a 10s timeout.
- Outcome: Exactly **1 request returned HTTP 201 Created**, and **19 requests returned HTTP 409 EVENT_FULL**. No overbooking occurred.

4. NFR-04 Data Persistence Evidence

The data persistence across container and API restarts was verified using the following documented procedure:

1. Seed database: `npm run db:seed` (5 users, 4 events, 1 registration).
2. Measure initial count: `users=5, events=4, registrations=1`.
3. Restart container: `docker compose restart db`.
4. Measure post-restart count: `users=5, events=4, registrations=1`.
5. Delta: `0` (100% data persistence preserved via PostgreSQL named volume).

5. NFR-05 Secret Isolation Evidence

Verification command:

```
git ls-files | grep '\.env'
```

Output:

```
.env.example
```

The real `.env` file containing database passwords is ignored by `.gitignore` and has never been committed.

6. GitHub Collaboration and Release History

All changes were integrated via reviewed Pull Requests on GitHub:

- `PR #10`: Skeleton, Prisma schema, Docker Compose (B)
- `PR #11`: SRS, baseline docs, SDS (B)
- `PR #12`: Contracts F-02, F-04, F-05, DoD (A)
- `PR #13` & `PR #14`: F-02 red and green (A)
- `PR #15`: F-01 Auth implementation (B, reviewed by A)
- `PR #16`: F-03 Browse events (B, reviewed by A)
- `PR #17`: F-06 Organizer view (B, reviewed by A)
- `PR #18`: Setup README & test DB documentation (A)
- `PR #19` & `PR #20`: F-02 HTTP & F-04 Concurrency (A, reviewed by B)
- `PR #21`: F-05 My registrations (B, reviewed by A)
- `PR #22`: Playwright E2E test suite (B, reviewed by A)
- `PR #23` & `PR #24`: Sprint 1 Release into `main` (Approved and merged by A)
- **Tag:** `v0.1.0-sprint1` attached to commit `ab9936e` on `main`.

End of Sprint 1 Deliverable Report.